

40dB

Sistema de Monitoreo y Gestión de Contaminación Acústica en Tiempo Real.

Integrantes del grupo

Joaquín Meléndez
Ignacio Saavedra

Profesor a cargo:
Cristian Calderón

Fecha de entrega:
21-06-2026

índice

Contents

Índice.....	Error! Bookmark not defined.
Introducción	3
Contexto del proyecto	5
2.1 Antecedentes y motivación.....	5
2.2 Origen del problema	5
2.3 Marco normativo en Chile	5
2.4 Estado del arte	6
Descripción del problema u oportunidad	9
Problema central.....	9
Análisis de causas y efectos	9
Descripción del mercado objetivo	11
Objetivo general y objetivos específicos	13
Situación inicial del cliente (AS-IS)	14
Proceso actual de gestión del ruido en una municipalidad	14
Puntos de dolor.....	14
Planificación	15
Servicios Cloud a utilizar (IaaS / PaaS / SaaS)	17
Conceptualización (solución propuesta – versión preliminar)	19
Alcance, supuestos y restricciones	21
Metodología de desarrollo y gestión del proyecto	23
Documentos y diagramas de diseño de la solución	26
Arquitectura de software y patrones de diseño	28
Tecnologías, lenguaje y dependencias (stack definitivo)	30
Estado de avance del desarrollo (evidencia de producto funcional)	35
Integraciones necesarias.....	37
Estrategia y enfoque de calidad.....	39
Plan de pruebas	41
Tipos de pruebas aplicadas	43
Herramientas y software utilizados.....	45
Ejecución de pruebas y resultados.....	47

Introducción

La contaminación acústica en zonas urbanas densamente pobladas se ha consolidado como un problema de salud pública con consecuencias documentadas sobre el sueño, la salud cardiovascular y el bienestar general de la población. Según datos del Ministerio del Medio Ambiente, más de 1,2 millones de habitantes del Gran Santiago están expuestos a niveles de ruido diurno por sobre las recomendaciones de la OCDE, cifra que asciende a 1,8 millones en horario nocturno. A pesar de la magnitud del problema, las herramientas con que cuentan las municipalidades chilenas para diagnosticarlo y actuar frente a él siguen siendo limitadas: mapas estáticos actualizados cada varios años, denuncias reactivas tramitadas caso a caso y mediciones puntuales en terreno que no permiten construir una visión continua del territorio.

El proyecto 40dB propone llenar ese vacío mediante una plataforma web de monitoreo colaborativo de ruido urbano que combina dos fuentes de datos complementarias: reportes ciudadanos enviados desde el navegador, con captura de decibeles desde el micrófono del dispositivo y geolocalización, y lecturas periódicas de sensores IoT basados en ESP32 instalados en puntos fijos de la comuna. Ambas fuentes alimentan un backend que cruza la información geográfica y temporal para asignar un nivel de confiabilidad a cada reporte, y un dashboard municipal que visualiza el resultado en un mapa de calor interactivo con filtros por zona, horario y tipo de fuente sonora. La solución está orientada a municipalidades como herramienta de gestión ambiental —en una primera etapa, a la Municipalidad de Maipú— permitiéndoles tomar decisiones basadas en datos empíricos continuos en lugar de depender exclusivamente de modelaciones teóricas o denuncias puntuales.

El presente informe documenta el diseño y el estado de avance del desarrollo del proyecto. Comienza por el contexto del problema, su marco normativo en Chile y el estado del arte de soluciones existentes a nivel nacional e internacional, para luego definir el problema central junto con su análisis de causas y efectos. Continúa con la descripción del mercado objetivo —municipalidades como cliente y vecinos como usuarios finales— y con el planteamiento de los objetivos generales y específicos que guían el desarrollo. A partir de allí se describe la situación inicial del cliente (AS-IS), la planificación del proyecto en seis sprints de dos semanas, los servicios cloud seleccionados (Render, Vercel, Supabase y HiveMQ Cloud) y la conceptualización preliminar de la solución (TO-BE), seguida del alcance, los supuestos y las restricciones que delimitan el trabajo. Las secciones posteriores profundizan en los aspectos de ingeniería: la metodología ágil adoptada por el

equipo y la distribución de responsabilidades, los documentos y diagramas de diseño que estructuran la solución (casos de uso, flujos AS-IS y TO-BE, mockups, modelo de datos y arquitectura), la arquitectura de software hexagonal lite junto con los patrones de diseño aplicados, el stack tecnológico definitivo y la configuración del servidor de producción paso a paso. Finalmente, se presenta el estado de avance del desarrollo con evidencia de producto funcional, las integraciones necesarias con servicios externos y una conclusión con los beneficios esperados y los próximos pasos del semestre.

Contexto del proyecto

2.1 Antecedentes y motivación

El ruido urbano es reconocido internacionalmente como un contaminante con impactos directos en la salud: trastornos del sueño, estrés, hipertensión y pérdida auditiva. La OMS estima la pérdida de al menos un millón de años de vida saludable anuales solo en Europa debido al ruido del tráfico. En Chile, la Superintendencia del Medio Ambiente (SMA) reporta que aproximadamente el 52% de las denuncias ambientales que recibe corresponden a contaminación acústica, y solo entre enero y marzo de 2025 ingresaron más de 1.100 denuncias por ruido, superando el promedio histórico del mismo período.

La motivación de este proyecto nace de la experiencia cotidiana de vivir en una ciudad ruidosa y de la constatación de que, pese a la existencia de normativa (DS 38/2011 del MMA, ordenanzas municipales), la fiscalización y el diagnóstico del problema carecen de herramientas tecnológicas accesibles y en tiempo real a nivel comunal.

2.2 Origen del problema

El problema se detectó por observación directa y experiencia personal: la percepción de niveles de ruido elevados en entornos urbanos cotidianos contrasta con la escasez de datos empíricos accesibles que permitan dimensionar y localizar el problema a escala de barrio o comuna. Actualmente, un vecino que sufre ruido molesto solo puede presentar una denuncia ante la SMA o la municipalidad, pero no tiene acceso a datos históricos ni en tiempo real de los niveles de ruido en su zona.

2.3 Marco normativo en Chile

La regulación del ruido en Chile se estructura en varios niveles. El **Decreto Supremo N°38/2011 del MMA** establece niveles máximos de emisión de ruido para fuentes fijas (industrias, comercio, construcción), con límites que varían según la zona territorial y el horario (diurno/nocturno). Por ejemplo, en zonas residenciales el límite nocturno es de 45 dB. Sin embargo, este decreto no regula fuentes móviles como el tránsito vehicular ni actividades propias de viviendas. Complementariamente, cada municipalidad puede dictar **ordenanzas locales** sobre ruidos molestos que cubren estos vacíos. A nivel nacional, se encuentra en elaboración una **Norma Primaria de Calidad Ambiental para Ruido**, que apunta a establecer estándares de calidad del aire acústico (no solo de emisión).

En febrero de 2026 se publicó el **Decreto Supremo N°14/2024 del MMA**, que actualiza el DS 38/2011 con exigencias más estrictas.

2.4 Estado del arte

En Chile:

- **Mapa de Ruido del Gran Santiago (MMA/UACH):** Modelación por software del ruido vehicular en 35 comunas, actualizado en 2023. Es una herramienta de diagnóstico estática que se actualiza cada varios años y no captura datos en tiempo real ni fuentes de ruido incidentales (construcciones, eventos, etc.).
- **Mapa de Denuncias de Ruido (SMA, 2025):** Plataforma pública lanzada en la plataforma IDE de la SMA que visualiza denuncias históricas por comuna desde 2013 a 2024. Es reactivo (solo registra lo que se denuncia) y no mide niveles de ruido.
- **Red de monitoreo piloto del MMA:** Estaciones con sonómetros en puntos específicos de Santiago para medir la calidad del aire en tiempo real. Cobertura muy limitada y no accesible como plataforma pública interactiva. Tiene un símil con 40 dB en el sentido de utilizar sensores que reportan continuamente un estado del ambiente, en este caso es calidad del aire reportada a través de datos, nosotros queremos reportar cantidad de ruido a través de un heatmap.
- **SOSAFE:** Red social ciudadana chilena con 2 millones de usuarios aprox y convenios con más de 30 municipios, Carabineros y Bomberos. Permite reportar incidentes geolocalizados en categorías como robos, emergencias y ruidos molestos. Su foco es seguridad general, no acústica: los reportes de ruido son textuales y se derivan a seguridad municipal, sin medir decibeles ni generar mapas de ruido. Modelo freemium con funciones limitadas en versión gratuita.

A nivel internacional:

- **The Noise App (Reino Unido):** App de RHE Global especializada en denuncias de ruido vecinal, operativa en UK, Países Bajos, Irlanda y Australia. Modelo B2G: gratuita para ciudadanos si su municipio está suscrito. Permite grabar clips de audio/video de 30 segundos como evidencia, admisible en tribunales. Es una herramienta de gestión de denuncias, no de monitoreo: no mide dB continuamente ni genera mapas acústicos.
- **Alerta Surco (Perú):** App municipal del distrito de Santiago de Surco (Lima), lanzada en 2023, con más de 65 mil usuarios. Botón de pánico para solicitar

serenazgo, bomberos o ambulancia con respuesta en 5 minutos, mostrando el recorrido de la unidad en mapa. Enfocada en seguridad y emergencias; anteriormente incluía categoría de ruidos molestos, pero fue eliminada. Alcance estrictamente distrital.

- **NoiseCapture (Francia):** Aplicación Android open-source para medición colaborativa de ruido ambiental. Genera mapas participativos con datos de smartphones. Solo Android, orientada a investigación, sin presencia activa en Latinoamérica.

Conclusión del análisis: No existe en Chile una plataforma que combine monitoreo ciudadano en tiempo real con sensores IoT fijos, orientada a municipalidades como herramienta de gestión. 40dB ocupa ese espacio.

Tabla de homologación:

Solución	Qué hace	Ventajas	Desventajas	Qué hará 40dB distinto
Mapa de Ruido MMA	Modela ruido vehicular por software en el Gran Santiago	Cobertura de 35 comunas, respaldo institucional	Estático, se actualiza cada año, no captura ruido incidental	Datos en tiempo real múltiples fuentes de ruido., granularidad comunal
Mapa Denuncias SMA	Visualiza denuncias históricas por comuna	Acceso público, cobertura nacional	Solo denuncias (reactivo), no mide dB, no es en tiempo real	Monitoreo proactivo con medición de dB + contexto denuncia ciudadana (proactivo + reactivo)
SOSAFE	Medio de	Acceso	Solo genera	Cruzar datos

	reportes en incidentes geolocalizados	público, reportes en tiempo real	reportes de usuarios, no contrasta con datos reales inmediatos.	reales con los reportes, será exclusivamente de ruido
The Noise App	App especializada en reportes de ruido	Permite grabar clips de audios para respaldar la denuncia.	No está disponible en Chile e incluye evidencia únicamente del denunciante.	Datos externos al usuario que reporta, dando mayor fiabilidad a la denuncia.

Descripción del problema u oportunidad

Problema central

Actualmente las municipalidades chilenas carecen de herramientas tecnológicas para monitorear el ruido urbano de forma continua y geolocalizada, lo que les impide diagnosticar zonas críticas con datos empíricos y tomar decisiones informadas de gestión ambiental.

Análisis de causas y efectos

El análisis se muestra presenta mediante este texto como también en un diagrama Ishikawa¹.

Causas principales:

1. **Tecnológicas:** No existen plataformas de monitoreo de ruido en tiempo real accesibles para municipios chilenos.
2. **Datos:** No hay recolección continua ni participativa de datos de ruido a nivel comunal. Los sensores profesionales (sonómetros clase 1) son costosos y de cobertura limitada
3. **Institucional:** Las municipalidades dependen de denuncias para detectar problemas de ruido. La fiscalización de la SMA opera de forma reactiva y con recursos limitados.
4. **Ciudadanía:** Los vecinos no tienen canales accesibles para reportar ruido con datos objetivos. Baja conciencia ciudadana sobre los efectos del ruido en la salud.

Efectos principales:

1. Diagnósticos ambientales incompletos o desactualizados que no reflejan la realidad acústica del territorio.
2. Decisiones de planificación urbana (zonificación, permisos de edificación, transporte) tomadas sin considerar el impacto acústico real.
3. Deterioro de la calidad de vida de los vecinos expuestos a niveles de ruido por sobre la norma, con efectos en salud (estrés, trastornos del sueño, hipertensión).

¹ Se adjunta un diagrama Ishikawa en el anexo * que grafica este capítulo.

4. Percepción ciudadana de desamparo frente al ruido, al no contar con datos ni herramientas para visibilizar el problema.

¿Qué parte ataca 40dB? La solución se enfoca en las causas tecnológicas y de datos: proveer una plataforma que genere datos empíricos de ruido de forma continua y participativa, entregando a la municipalidad un dashboard con información accionable y al vecino un canal para contribuir y visualizar el estado acústico de su entorno.

Descripción del mercado objetivo

Cliente principal: Municipalidades

La municipalidad es quien toma la decisión de adoptar y eventualmente financiar la herramienta. Su necesidad es contar con datos empíricos de ruido para fiscalizar ordenanzas, priorizar intervenciones y reportar a la SMA. Su dolor actual es depender de denuncias puntuales y no tener visibilidad continua del problema.

Perfil del cliente

Quién es: Dirección de Medio Ambiente o Departamento de Fiscalización de una municipalidad.

Necesidad: Monitorear niveles de ruido por zonas, identificar focos críticos, generar reportes para la toma de decisiones.

Escenario de uso: El encargado ambiental accede al dashboard de 40dB, revisa el heatmap de la comuna, identifica zonas con niveles sobre la norma, y prioriza fiscalizaciones o intervenciones.

Usuario final: Vecinos de la comuna

El vecino es quien usa la aplicación para reportar ruido y consultar el estado acústico de su barrio. No paga directamente, pero su participación es esencial para alimentar el sistema de datos.

Perfil del usuario

Quién: Residente de la comuna (cualquier edad, con acceso a un navegador web).

Necesidad: Saber cuánto ruido hay en su zona, poder reportar situaciones de ruido excesivo con datos objetivos.

Escenario de uso: El vecino abre la app web en su celular, el micrófono captura automáticamente el nivel de dB por unos segundos, el usuario selecciona el tipo de fuente de ruido (tráfico, construcción, local nocturno, etc.) y envía el reporte geolocalizado.

Modelo de relación

El modelo es **B2G** (Business to Government): 40dB se ofrece a la municipalidad como herramienta de gestión, y la municipalidad habilita el acceso público para sus vecinos. El vecino es el usuario que genera datos; la municipalidad es el cliente que consume la información procesada.

Objetivo general y objetivos específicos

Objetivo general

Desarrollar una plataforma web de monitoreo colaborativo de ruido urbano que permita a la Municipalidad de Maipú visualizar en tiempo real los niveles de contaminación acústica por zonas, combinando reportes ciudadanos con datos de sensores IoT, para facilitar la toma de decisiones en gestión ambiental.

Objetivos específicos

1. **Diseñar e implementar** una aplicación web que permita a los vecinos reportar niveles de ruido desde su navegador, capturando dB mediante el micrófono del dispositivo junto con la geolocalización y el tipo de fuente sonora.
2. **Desarrollar** un backend con API REST que reciba, almacene y procese los datos de reportes ciudadanos y lecturas de sensores IoT, y los exponga para su visualización en un mapa de calor interactivo.
3. **Construir** un prototipo funcional de sensor IoT, potencialmente con ESP32 y un sensor de sonido, capaz de enviar lecturas periódicas de niveles de ruido al backend, y diseñar el mecanismo de cruce de datos entre sensores fijos y reportes ciudadanos para establecer niveles de confiabilidad.
4. **Implementar** un dashboard municipal que permita visualizar el heatmap de ruido, filtrar por zona, horario y tipo de fuente, y generar reportes exportables con los datos recopilados.

Situación inicial del cliente (AS-IS)²

Proceso actual de gestión del ruido en una municipalidad

Hoy, el flujo de gestión del ruido en una municipalidad chilena típica funciona de la siguiente manera:

1. **Un vecino percibe ruido molesto** en su barrio (construcción, local nocturno, tráfico, etc.).
2. **El vecino presenta una denuncia** a través de los canales disponibles de la municipalidad (fono municipal, correo electrónico, presencial) o directamente ante la SMA.
3. **La municipalidad asigna un inspector** que se desplaza al lugar con un sonómetro portátil para realizar una medición puntual.
4. **El inspector mide en terreno**, compara con los límites de la ordenanza municipal y/o el DS 38, y elabora un informe.
5. **Si se detecta infracción**, se inicia un procedimiento sancionatorio ante el Juzgado de Policía Local o se deriva a la SMA.
6. **No existe registro centralizado** ni georreferenciado de las mediciones ni de los niveles de ruido históricos por zona. La información queda dispersa en informes individuales.

Puntos de dolor

Reactivo: Solo se actúa cuando hay denuncia; no hay monitoreo preventivo.

Puntual: La medición es de un momento y lugar específico; no refleja patrones temporales ni espaciales.

Sin datos históricos accesibles: No hay forma de comparar niveles de ruido entre zonas ni de ver tendencias en el tiempo.

Costoso en recursos humanos: Cada denuncia requiere desplazamiento de un inspector con equipamiento.

Opaco para el vecino: El denunciante no tiene acceso a datos ni seguimiento del estado acústico de su entorno

² Se adjunta un diagrama AS-IS que grafica este capítulo. Se encuentra en el anexo "Diagrama AS IS.png" adjunto en el envío.

Planificación

El proyecto se desarrollará siguiendo una metodología ágil adaptada de Scrum, organizada en 6 sprints de 2 semanas a lo largo de 12 semanas especificados en una Carta Gantt³. Cada sprint termina con un entregable funcional que permite validar avances de forma incremental, obtener una retroalimentación temprana y ajustar el alcance según lo aprendido.

Sprint	Semanas	Objetivo	Entregable Funcional
S0 — Kickoff	1 - 2	Fundamentos técnicos: definir arquitectura, levantar requerimientos, crear repositorios y desplegar entornos base en la nube.	Frontend y backend desplegados y conectados con un endpoint dummy; base de datos creada con modelo inicial.
S1 — Reporte	3 - 4	Flujo completo mínimo: autenticación + envío de un reporte manual + visualización en el mapa.	Un vecino autenticado puede enviar un reporte geolocalizado y verlo como punto en el mapa.
S2 — Audio + heatmap	5 - 6	Captura de dB reales desde el navegador y visualización como mapa de calor.	El reporte mide automáticamente el nivel de dB; el mapa muestra un heatmap en lugar de pines individuales.
S3 — IoT integrado	7 - 8	Integración del prototipo de sensor físico con el backend.	Prototipo ESP32 + Sensor de audio enviando datos reales vía HiveMQ, visibles en el heatmap junto a los reportes ciudadanos.
S4 — Cruce + dashboard	9 - 10	Algoritmo de confiabilidad y panel para la municipalidad.	Dashboard municipal con filtros (zona, horario, tipo), métricas agregadas y reportes validados

³ Revisar Anexo “Carta Gantt.xlsx” adjuntado en el envío del proyecto.

			contra sensores con índice de confiabilidad.
S5 — Refinamiento y entrega	11 - 12	Pulido, exportación, documentación y presentación final.	Producto terminado desplegado, documentación técnica y demo lista.

Servicios Cloud a utilizar (IaaS / PaaS / SaaS)

La solución se desplegará en la nube para garantizar disponibilidad, escalabilidad y acceso remoto tanto para los sensores IoT como para los usuarios de la plataforma. Se utilizará Render, Vercel y Supabase, aprovechando sus capas gratuitas (Free Tier) dado que el presupuesto es simulado/académico.

Componente	Servicio	Tipo Cloud	Justificación
Servidor de aplicación (Backend FastAPI)	Render	PaaS	Render abstrae la infraestructura: despliegue directo desde Git, SSL automático, sin administrar servidor.
Base de datos	Supabase (PostgreSQL con PostGIS)	PaaS	Base de datos gestionada con backups automáticos, escalado y soporte nativo para extensiones geoespaciales. Libera al equipo de administrar el motor de BD.
Almacenamiento	Supabase Bucket	IaaS	Almacenamiento de objetos para exportaciones de reportes, respaldos de datos y archivos estáticos del frontend.
Frontend (Vue3)	Vercel	PaaS	Despliegue automático desde Git con CDN global, integración nativa con frameworks JS.
Recepción de datos IoT	HiveMQ	PaaS	Servicio gestionado para recibir mensajes MQTT desde los sensores ESP32. Maneja autenticación de dispositivos,

			enrutamiento de mensajes y escalado automático.
Autenticación	Supabase Auth	SaaS	Servicio de identidad gestionado por Supabase. Permite delegar a esta plataforma el proceso de autenticación y autorización de los usuarios.

Conceptualización (solución propuesta – versión preliminar)⁴

Descripción general (TO-BE)

40dB es una plataforma web de monitoreo colaborativo de ruido urbano que opera bajo un modelo híbrido de recolección de datos: reportes ciudadanos desde el navegador y lecturas automáticas de sensores IoT fijos. Los datos se procesan y visualizan en un mapa de calor interactivo accesible tanto para vecinos como para el equipo técnico municipal.

El flujo con 40dB implementado sería:

1. **Vecinos** abren la app web, el micrófono del dispositivo captura el nivel de dB durante unos segundos, el usuario categoriza el tipo de ruido y envía el reporte geolocalizado.
2. **Sensores IoT** instalados en puntos estratégicos envían lecturas de dB cada 5 minutos al backend vía MQTT.
3. **El backend** recibe ambos flujos, los almacena con coordenadas y timestamp, y ejecuta el cruce de datos: los reportes ciudadanos que caen dentro de un radio establecido de un sensor fijo se comparan con la lectura del sensor en el mismo rango horario, asignándoles un índice de confiabilidad.
4. **El heatmap** se genera interpolando todos los puntos de datos (sensores + reportes ponderados por confiabilidad) y se visualiza sobre un mapa de la comuna.
5. **El dashboard municipal** permite filtrar por zona, rango horario, tipo de fuente y período, y exportar reportes en PDF/CSV.

Funcionalidades principales

1. **Reporte ciudadano con captura de dB:** Interfaz web que utiliza la Web Audio API para medir el nivel de presión sonora desde el micrófono del dispositivo, combinado con geolocalización (GPS del navegador) y selección del tipo de fuente de ruido.
2. **Mapa de calor interactivo (heatmap):** Visualización en Leaflet.js sobre OpenStreetMap de los niveles de ruido por zona, actualizado en tiempo real con los datos entrantes. Accesible para vecinos y municipalidad.

⁴ Revisar diagrama de arquitectura en el anexo “Diagrama Arquitectura.png” adjuntado en el envío.

3. **Ingesta de datos IoT:** Endpoint MQTT/HTTP para recibir lecturas de sensores ESP32 con nivel de dB, timestamp y coordenadas fijas registradas en el sistema.

4. **Motor de cruce y confiabilidad:** Algoritmo que compara reportes ciudadanos con lecturas de sensores cercanos para asignar un puntaje de confiabilidad a cada reporte, ponderando su peso en el heatmap.

5. **Dashboard municipal:** Panel con filtros avanzados (zona, hora, fecha, tipo de fuente), estadísticas agregadas (promedio dB por zona, horarios críticos, evolución temporal) y exportación de reportes.

6. **Historial y tendencias:** Almacenamiento de datos históricos que permite visualizar la evolución del ruido por zona en el tiempo, comparar períodos y detectar patrones.

Consideraciones de calidad y seguridad

Privacidad: Los reportes ciudadanos son anónimos. No se almacena audio, solo indicadores acústicos (dB) y metadatos.

Validación de datos: Los reportes pasan por validación básica (rango de dB plausible, geolocalización dentro de la comuna, rate limiting por IP/dispositivo para evitar spam).

Seguridad IoT: Los sensores se autentican ante HiveMQ mediante certificados, evitando la inyección de datos falsos.

Respaldo: La base de datos en PostgreSQL gestionada en Supabase cuenta con backups automáticos diarios.

Autenticación: Tanto los funcionarios municipales como los vecinos que envían reportes deben iniciar sesión con una cuenta de con su correo y contraseña. El backend valida el token con Supabase. Este enfoque cumple un doble propósito: evita gestionar directamente las cuentas, sino que mediante Supabase Auth y aprovecha la seguridad de Supabase y previene el spam y los reportes falsos al asociar cada reporte a una identidad verificada, permitiendo además aplicar rate limiting por usuario y detectar comportamientos anómalos. Los roles se diferencian en el backend: los vecinos solo pueden crear reportes y visualizar

el mapa público, mientras que los funcionarios municipales acceden al dashboard completo con métricas y exportación de datos.

Alcance, supuestos y restricciones

Alcance (incluye)

- Aplicación web funcional para reporte ciudadano con captura de dB y geolocalización.
- Backend con API REST para ingesta de reportes y datos de sensores.
- Mapa de calor interactivo sobre la comuna de Maipú.
- Prototipo de al menos 1 sensor IoT (ESP32 + sensor de audio) enviando datos al backend.
- Dashboard municipal con filtros básicos y exportación de datos.
- Mecanismo de cruce de datos sensor/reporte con índice de confiabilidad.
- El sensor cuando este encendido debe enviar información al broker cada 10 segundos.

No alcance (excluye)

- Aplicación móvil nativa (iOS/Android). La solución es web responsive.
- Despliegue de una red extensa de sensores. Se prototipa con 1–2 unidades.
- Integración directa con sistemas de la SMA o del MMA.
- Calibración profesional certificada de las mediciones (el proyecto genera datos indicativos, no mediciones normativas con validez legal).
- Gestión de denuncias formales ni flujos sancionatorios.

Entregables

- Código fuente del frontend, backend y firmware del sensor.
- Base de datos poblada con datos de prueba y datos reales del piloto.
- Documentación técnica (arquitectura, API, modelo de datos).
- Informe final del proyecto.
- Presentación de defensa del prototipo.

Supuestos

- Los navegadores modernos permiten acceso al micrófono del dispositivo vía Web Audio API con el consentimiento del usuario.
- Se dispondrá de al menos un punto con acceso a WiFi y electricidad para instalar el sensor piloto en Maipú.
- Los usuarios del piloto contarán con dispositivos con GPS y micrófono funcional.
- HiveMQ Free Tier será suficiente para el volumen de datos del prototipo.

Restricciones

- **Tiempo:** 12 semanas de desarrollo.
- **Equipo:** 2 integrantes.
- **Presupuesto:** Simulado/académico. Hardware IoT con costo real estimado bajo (10.000 -- 15.000 CLP por sensor).
- **Precisión de mediciones:** Los micrófonos de smartphones no están calibrados profesionalmente; los datos son indicativos y comparativos, no normativos.
- **Cobertura:** El piloto se limita a la comuna de Maipú, con datos representativos, pero no exhaustivos.

Metodología de desarrollo y gestión del proyecto

El desarrollo de 40dB se gestiona con una metodología ágil adaptada de scrum, coherente con la planificación presentada previamente: La idea de 6 sprints de 2 semanas a lo largo de 12 semanas, cada uno cerrado con una entrega funcional. Scrum se eligió porque el proyecto tiene requerimientos que maduran a medida que se construye el prototipo y un enfoque iterativo permite entregar valor de forma incremental, validar supuestos temprano y ajustar el alcance sin comprometer la fecha de cierre. Dado que el equipo es de solo 2 integrantes, se aplica una variante liviana: no existe un scrum master dedicado y, dentro de cada sprint, el trabajo se organiza con un flujo tipo kanban sobre un tablero de tareas.

Trabajo semana a semana: planificación, seguimiento y control

- **Planificación del sprint:** Al inicio de cada sprint de 2 semanas, el equipo selecciona elementos del backlog de producto, los descompone en tareas y estima su esfuerzo. El objetivo del sprint queda definido por el entregable funcional comprometido.
- **Seguimiento:** Las tareas se gestionan en un tablero de **Trello** con columnas Backlog, “nombrarlas”. Al ser un equipo de 2 personas, las reuniones diarias se reemplazan por *sincronizaciones asíncronas* breves y al menos tres reuniones de coordinación presenciales o remotas por semana
- **Control de versiones:** El código se gestiona en repositorios Git con ramas principales main y dev. Todo cambio se integra mediante Pull Request revisado por el otro integrante antes de hacer merge a dev.
- **Cierre del sprint:** Cada sprint termina con una revisión (demostración del entregable funcional) y una retrospectiva en la que se identifica que funcionó, que no, y que ajustar para el siguiente ciclo.

Roles y responsabilidades del equipo

El equipo está compuesto por 2 integrantes, ambos en el rol de desarrollador. La distribución de responsabilidades principales, sin perjuicio de que las decisiones de arquitectura, revisión de código, pruebas y documentación se asumen de forma conjunta, es la siguiente:

Área de responsabilidad	Responsabilidad principal	Alcance
Backend, base de datos e infraestructura cloud	Joaquín Meléndez	API REST en FastAPI, modelo de datos PostGIS, despliegue en Render y Supabase.
Frontend y subsistema IoT	Ignacio Saavedra	Aplicación web en Vue 3, mapa de calor, firmware del sensor ESP32 e ingesta MQTT.
Responsabilidades compartidas	Ambos integrantes	Definición de arquitectura, revisión de Pull Requests, pruebas, documentación técnica y preparación de la presentación.

La asignación corresponde a la responsabilidad principal de cada integrante; ambos pueden colaborar transversalmente cuando un sprint lo requiera.

Definición de Hecho (Definition of Done) y criterios de calidad

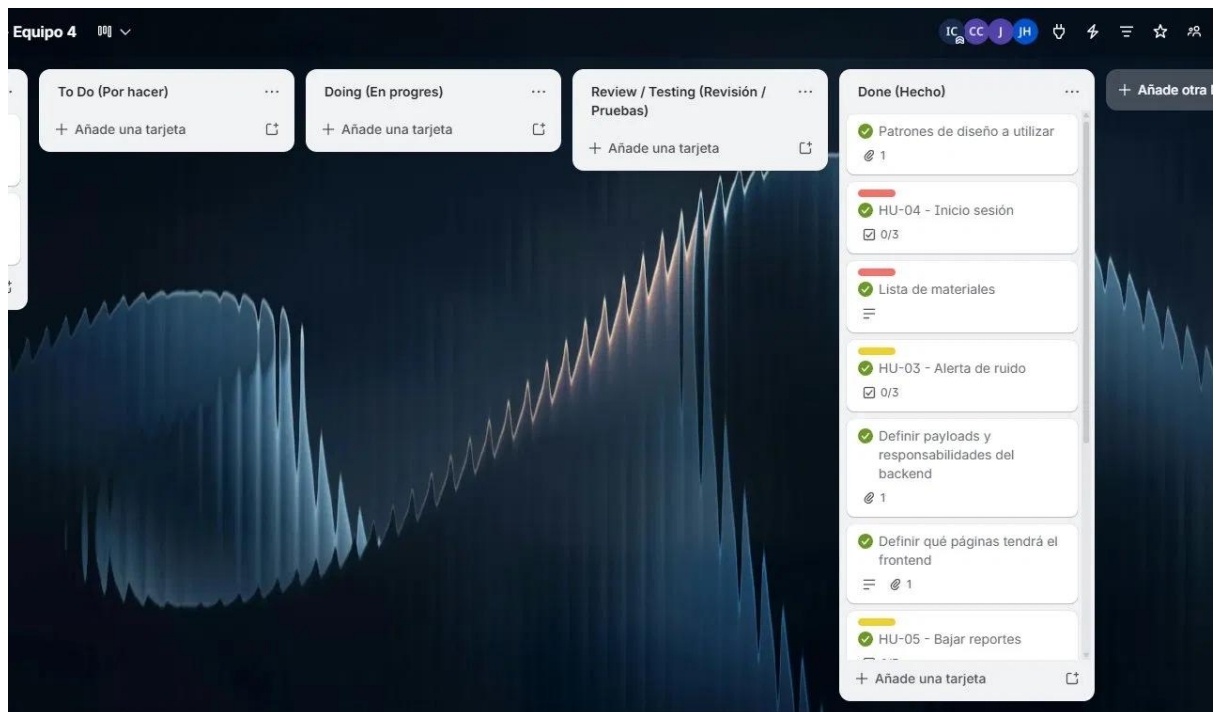
Una tarea o funcionalidad se considera hecha cuando se cumple todas las siguientes condiciones:

- El código implementa los criterios de aceptación de la historia de usuario correspondiente.
- Fue revisada y aprobada en un Pull Request por el otro integrante.
- Incluye pruebas mínimas cuando se aplica, o queda registrada como deuda técnica si la cobertura se posterga.
- Está integrada en la rama dev y desplegada en el entorno correspondiente.

Como criterios de calidad mínimos transversales se exigen: validación de entradas, manejo explícito de errores, control de acceso por rol, ausencia de secretos en el repositorio.

Hitos del proyecto: los hitos corresponden a los entregables funcionales de cada sprint (S0 a S5) descritos en el capítulo de planificación, cuyas fechas detalladas se encuentran en el anexo “carta gantt”⁵ (<https://docs.google.com/spreadsheets/d/1jX1qrzsDmBgooaQ3M7IFDdhv6EdEumLV/edit?usp=sharing&ouid=107429315056806719482&rtpof=true&sd=true>)

⁵ Revisar Anexo “Carta Gantt.xlsx” y “Tablero Trello.pdf” adjuntado en el envío del proyecto.



Ejemplo de Trello

Documentos y diagramas de diseño de la solución

Para guiar el desarrollo, el equipo elaboró un conjunto de documentos y diagramas de diseño. A continuación, se describe, para cada uno, su propósito, que representa y cómo se relaciona con el problema y los objetivos del proyecto. Los diagramas se adjuntan como anexos de esta entrega (Adjuntados en el zip de Diagramas y carta gantt.zip)

1. Diagrama de flujo del usuario (User Flow) y BPMN del proceso

- a. **Propósito:** representar paso a paso la interacción de los usuarios con el sistema
- b. **Que representa:** por un lado, el diagrama AS-IS, que modela el proceso actual de gestión del ruido en la municipalidad; por otro, los flujos TO-BE: el vecino genera un reporte, el vecino consulta el heatmap, el funcionario que gestiona una alerta y el flujo de inicio de sesión.
- c. **Relación con el problema y objetivos:** El diagrama AS-IS evidencia los puntos de dolor del proceso reactivo actual, mientras que los flujos TO-BE muestran como 40dB los resuelve, sirviendo de guía directa para implementar las vistas y la navegación de la aplicación.

2. Mockups de las pantallas principales

- a. **Propósito:** definir la estructura y disposición de la interfaz antes de programarla.
- b. **Que representa:** las pantallas del producto, la landing pública, el inicio de sesión, el home del vecino, la bandeja del funcionario con el modal del cambio de estado, el home de administrador y la vista de heatmap con filtros y leyenda.
- c. **Relación con el problema y objetivos:** los wireframes son la base del sistema de diseño (tokens y componentes base reutilizable) y se materializan en las vistas implementadas, aseguran una experiencia coherente y accesible para distintos públicos, desde el vecino hasta el funcionario.

3. Modelo de datos

- a. **Propósito:** definirla estructura de persistencia de la información.
- b. **Que representa:** las entidades **comuna**, **tipo_estado**, **usuario**, **sensor**, **lectura**, **reporte** e **historial_estado**, con sus relaciones principales: un usuario extiende 1:1 la identidad de autenticación, un sensor genera muchas lecturas (1:N), un reporte puede vincularse a una lectura como evidencia, y

un reporte acumula un historial de estados. Las entidades geográficas incorporan una columna de ubicación con PostGIS.

- c. **Relación con el problema y objetivos:** este modelo habilita el cruce geo-temporal entre reportes ciudadanos y lecturas de sensores, que es el núcleo diferenciador de la solución.

4. Diagrama de arquitectura de alto nivel

- a. **Propósito:** Mostrar los componentes del sistema y como se comunican entre si.
- b. **Que representa:** el frontend (aplicación web Vue desplegada en Vercel), el backend (API FastAPI en Render), la base de datos (PostgreSQL con PostGIS en Supabase), el broker MQTT (HiveMQ Cloud), los sensores ESP32 y el servicio de autenticación.
- c. **Relación con el problema y objetivos:** evidencia como el modelo híbrido de datos (reportes más telemetría IoT) se realiza técnicamente y como cada componente contribuye al objetivo de entregar un mapa de ruido.

5. Diagrama de casos de uso e historias de usuario

- a. **Propósito:** delimitar el alcance funcional del sistema identificando actores y las acciones que cada uno puede realizar.
- b. **Que representa:** los actores **Vecino(ciudadano)**, **Funcionario municipal** y **Administrador**, junto con las historias que estructuran el desarrollo.
- C. Relación con el problema y objetivos:** cada historia de usuario se vincula directamente con un objetivo específico; el reporte ciudadano, el dashboard municipal y la visualización del heatmap y asegura que el desarrollo cubra todos los actores del modelo B2G.⁶

⁶ Revisar los flujos TO-BE en los anexos "Carpeta Flujos TO-BE".
Mockup inicial en anexo "40dB — Prototipo Frontend.pdf".
Modelo de datos en anexo "Modelo de datos.pdf".
Diagrama arquitectura en anexo "Diagrama C4 - 40dB.png"
El proceso AS-IS se encuentra en el anexo "Diagrama AS IS.png".

Arquitectura de software y patrones de diseño

Arquitectura general

40db adopta una arquitectura **cliente-servidor multicapa**. La capa de presentación es una aplicación web construida en Vue 3; la capa de aplicación y lógica de negocio es un backend en FastAPI; y la capa de datos es una base de PostgreSQL con la extensión geoespacial PostGIS. A estas se suma un canal sincrónico de ingesta IoT sobre el protocolo MQTT, que alimenta el sistema con la telemetría de los sensores.

El desafío central de la arquitectura es la asimetría de tráfico: el frontend genera peticiones HTTP esporádicas, mientras que los sensores producen un flujo MQTT constante. La solución es un proceso único con doble punto de entrada: un solo proceso FastAPI atiende las peticiones HTTP y, mediante el ciclo de vida de la aplicación, levanta de forma asíncrona un suscriptor MQTT.

Arquitectura interna del backend: hexagonal lite

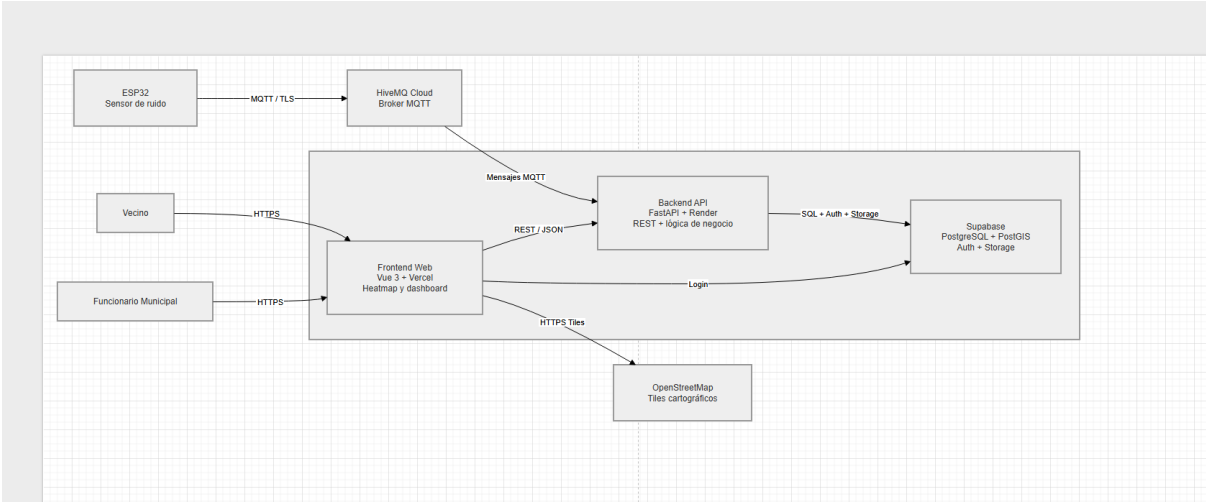
El backend sigue en **arquitectura hexagonal (Puertos y Adaptadores) en versión liviana**. Los puertos y adaptadores se aplican únicamente en los bordes de entrada/salida, el acceso a Supabase, la mensajería MQTT y la API HTTP, mientras que el núcleo es una orquestación delgada, dado que gran parte de la lógica de negocio (el cruce geo-temporal) reside en funciones de PostgreSQL. La regla de dependencia es estricta: siempre apuntan hacia dentro. Las capas api y mqtt dependen de la capa de aplicación (casos de uso), que depende de los puertos del dominio, que no depende de nada. La infraestructura implementa esos puertos. Esto mantiene el dominio testeable y desacoplado de la tecnología concreta.

Se implementó una variante “lite” porque una arquitectura hexagonal completa, con inyección de dependencia externa, sería sobre ingeniería para un prototipo desarrollado por 2 personas en 12 semanas; el mecanismo de dependencias propio de FastAPI es suficiente.

Arquitectura interna del frontend

⁷El frontend usa una arquitectura **basada en componentes** con una estratificación de **vistas finas, compostables y servicios**. Las vistas orquestan y muestran, los compostables encapsulan lógica reactiva reutilizable (por ejemplo, la carga de datos del heatmap con auto-refresco).

⁷ Se ocupa el concepto de hexagonal lite haciendo referencia al concepto adaptado de clean code pero contextualizado a nuestro proyecto ya que no requeríamos de arquitectura completa



Tecnologías, lenguaje y dependencias (stack definitivo)

A continuación, se presenta el stack tecnológico definitivo del proyecto. Respecto de la versión preliminar de servicios cloud, la persistencia y la autenticación se consolidaron en **Supabase**, que provee PostgreSQL gestionando con PostGIS y un servicio de autenticación integrado, reduciendo la carga operativa del equipo.

Frontend

- **Lenguaje y framework:** Typescript con **Vue 3**, empaqueta con **Vite**.
- **Librerías clave:** Pinia (gestion de estado), Vue Router en modo history (Enrutamiento), leaflet más leaflet heat (Mapa de calor), jsPDF, jspdf-autotable y html2canvas (exportacion de reportes).
- **Calidad:** ESLint y oxlint mas prettier (linting y formato); Vitest con @vue/test-utils (pruebas unitarias) y Cypress (pruebas end-to-end).

Backend

- **Lenguaje y framework:** Python con FastAPI servido por uvicorn
- **Librerías clave:** supabase, paho-mqtt (cliente MQTT del ingestor), pydantic-settings (configuración por variables de entorno), pyjwt (verificación local de JWT emitidos por Supabase Auth), python-ulid (generación de correlation ids para trazabilidad de request).
- **Base de datos:** PostgreSQL con la extensión de PostGIS.
- **Mensajería:** HiveMQ Cloud como broker MQTT.

Subsistema IoT

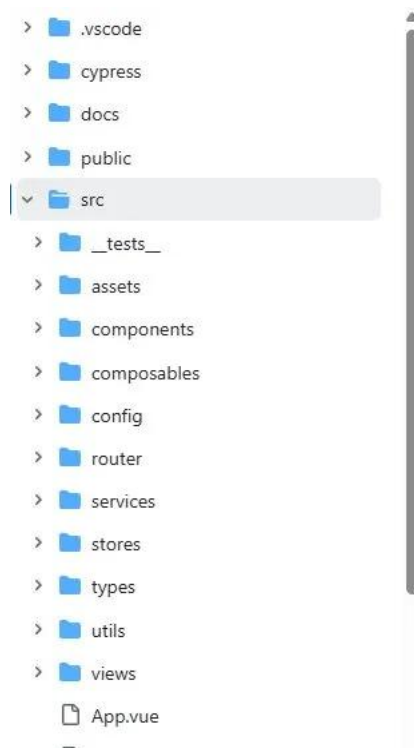
- **Hardware:** microcontrolador ESP32 con un sensor de sonido.
- **Firmware:** desarrollado en C/C++ (Entorno Arduino), publicando lecturas vía MQTT sobre TLS.

Estándares

- **Estructura de carpetas:** el backend sigue la estructura hexagonal lite (dominio, aplicacion, infraestructura y api); el frontend se organiza por responsabilidad (Componentes comunes, de layout y de mapa; composables; servicios; stores; tipos; y vistas por rol), con el alias @ apuntando a src/.
- **Linters y formato:** ESLint, oxlint y Prettier en el frontend; estilo PEP 8 en el backend. La API expone sus endpoints bajo el prefijo /api/v1.
- **Convención de commits:** mensajes en minúsculas y en español, con prefijos feat:, fix:, chore: y add:.

Repositorios y flujo de trabajo

El proyecto se gestiona en dos repositorios Git (frontend y backend). Cada repositorio mantiene las ramas principales **main** (estable) y **dev** (integración). El flujo de trabajo es: desarrollo en una rama de funcionalidad, apertura de Pull Request, revisión por el otro integrante, merge a dev y, integración a main.



Estructura de frontend.

☐	 Sync dev → main: landing, dashboard admin, home vecino, home funciona... ✓
	#20 by JoaquinMelendez (Collaborator) was merged 4 days ago
☐	 Sync dev → main: landing, dashboard admin, home vecino, home funciona... ✓
	#19 by JoaquinMelendez (Collaborator) was merged 4 days ago
☐	 add: initial CLAUDE.md to persist the directions to backend documenta... ✓
	#18 by JoaquinMelendez (Collaborator) was merged 4 days ago • Approved
☐	 Sync dev → main: landing, dashboard admin, home vecino, home funcionario, docs HU ✓
	#17 by dell000 (Owner) was merged 4 days ago • Approved
☐	 docs: diagramas de flujo de usuario por rol
	#16 by dell000 (Owner) was merged 2 weeks ago • Approved
☐	 feat: home del funcionario con gestion de alertas HU-002
	#15 by dell000 (Owner) was merged 5 days ago • Approved
☐	 feat: home del vecino con heatmap embebido y formulario de reporte
	#14 by dell000 (Owner) was merged 5 days ago • Approved

Ejemplo de PR's y commit

Configuración de servidor de producción (paso a paso)

Plataforma y componentes

- **Backend (FastAPI):** desplegado en **Render** (PaaS), con SSL y dominio gestionados automáticamente.
- **Base de datos y autenticación: Supabase**, que provee PostgreSQL gestionado con la extensión PostGIS y el servicio Supabase Auth.
- **Frontend (Vue/Vite):** desplegado en **Vercel** (PaaS) con CDN global.
- **Broker MQTT: HiveMQ Cloud**, en su capa gratuita, con TLS obligatoria (puerto 8883)
- **Almacenamiento de objetos:**

Requisitos del servidor

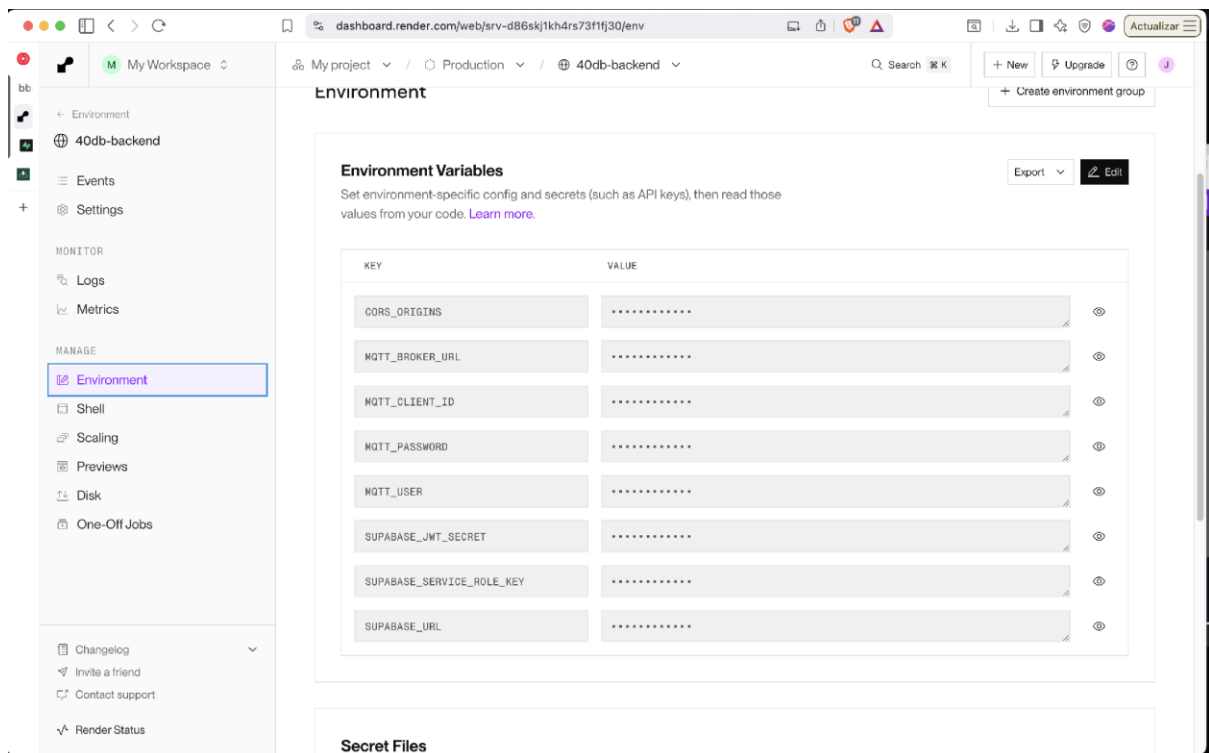
- **Runtime:** Python para el back, Node 20+ para el build del frontend (Vite)
- **Variables de entorno del backend:**
 - **Para Supabase / Frontend**
 - SUPABASE_URL
 - SUPABASE_SERVICE_ROLE_KEY
 - SUPABASE_JWT_SECRET
 - CORS_ORIGINS
 - **MQTT**
 - MQTT_BROKER_URL
 - MQTT_USER
 - MQTT_PASSWORD
 - MQTT_CLIENT_ID
- **Variables del entorno del frontend:**
 - VITE_API_URL
 - anon_key (supabase)
- **Puertos:**
 - Backend expone 8000 (Render publica vía 443/SSL); broker MQTT en 8883 (TLS).
- **Permisos:**
 - service_role_key

Instrucciones paso a paso

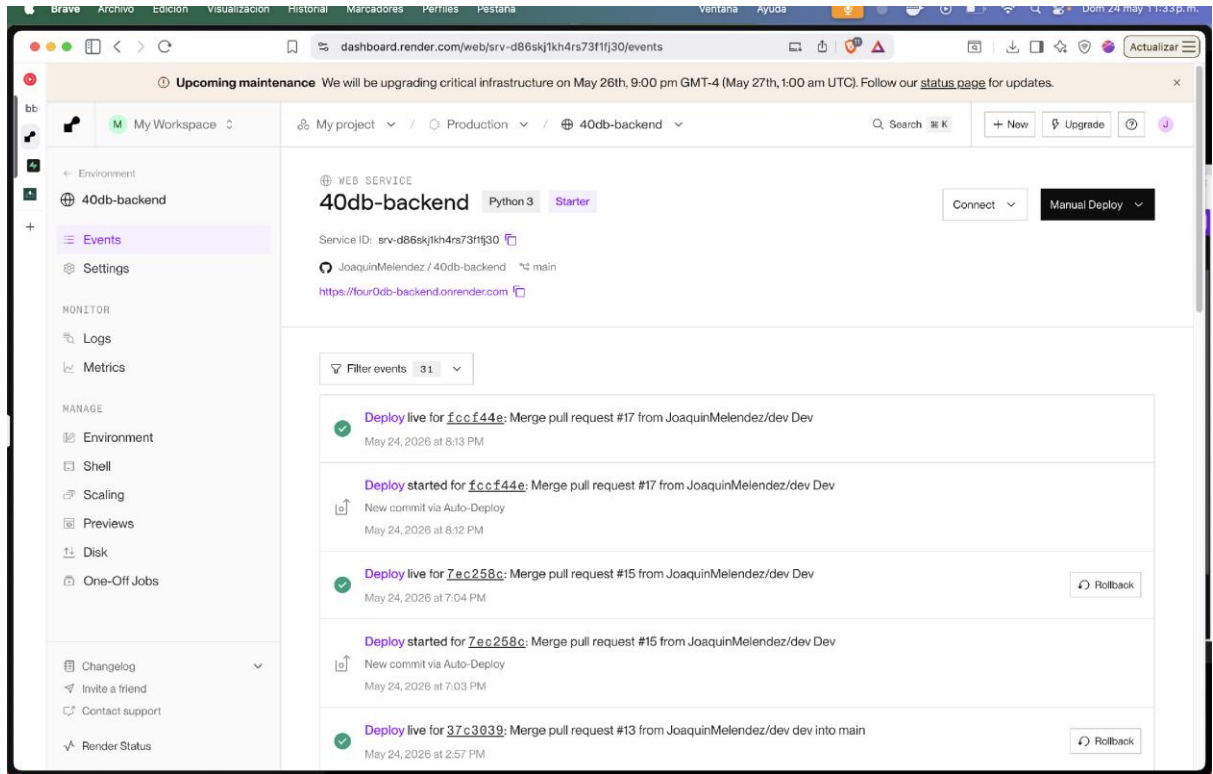
- **Provisión de DB:** crear proyecto Supabase, habilitar PostGIS, correr migraciones (supabase db reset local → supabase db push).

- Instalación de lenguaje y dependencias: `python -m venv .venv, pip install -r requirements.txt`.
- Configuración de secretos: cargar variables en el panel de Render (backend) y Vercel (frontend).
- Despliegue: push a main dispara el build automático en Render y Vercel.
- Configuración HiveMQ: crear cluster gratuito, usuario backend-subscriber (SUBSCRIBE 40db/#) y usuario 40db-sensor (PUBLISH 40db/sensores/+/lectura), descargar cert CA (ver `iot.md §18`).
- Logs y monitoreo: logs nativos de Render para el backend; `/health/live` y `/health/ready` para el orquestador (ver `api.md API2` / `backend.md ADR 06`).

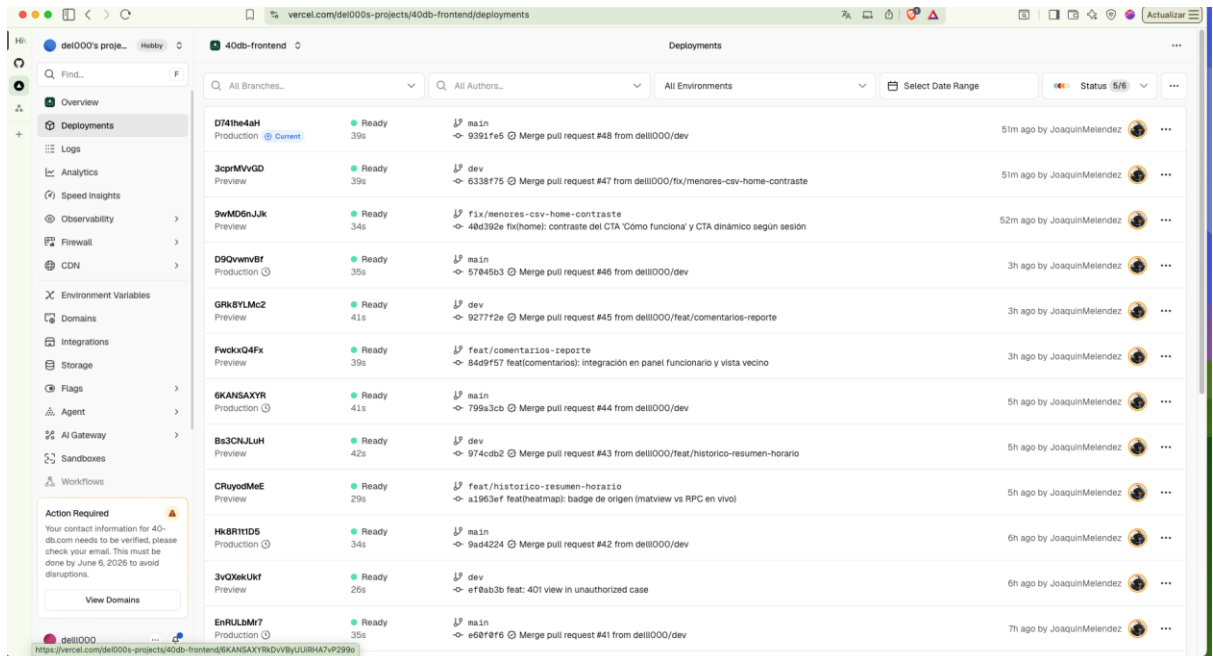
Evidencias:



- <https://four0db-backend.onrender.com>



Frontend:



Estado de avance del desarrollo (evidencia de producto funcional)

El desarrollo del prototipo se encuentra en un estado avanzado: el sistema **opera de extremo a extremo con datos reales**. Un sensor físico basado en **ESP32** captura niveles de ruido y publica sus lecturas mediante el broker **MQTT**; el **backend** las ingiere y las persiste en la base de datos **Supabase/PostGIS**; y el **frontend** consume esa información a través de la API REST y la visualiza en el mapa de calor y en las vistas por rol. Actualmente hay un único sensor ESP32, por lo que el heatmap refleja las lecturas de ese dispositivo; arquitectura admite N sensores y la incorporación de más unidades es una tarea de despliegue de hardware, no de desarrollo de software. La siguiente tabla detalla las funcionalidades y su estado.

Funcionalidad	Estado	Descripción
1. Sistemas de diseño	Implementada	Tokens de diseño en variables CSS y componentes base reutilizables en el frontend
2. Routing con control de acceso por rol	Implementada	Rutas con carga diferida, guards de autenticación y rol, y restauración de la sesión antes de montar el router.
3. Mapa de calor interactivo (heatmap)	Implementada	Render con Leaflet y leaflet heat sobre OpenStreetMap, centrado en Maipú; filtros reactivos, auto-refresco, leyenda de bandas dB y banner de alerta sostenida. Consume las lecturas reales servidas por el backend
4. Exportación de reportes del heatmap	Implementada	Generación 100% en cliente en formatos PNG, CSV y PDF, este último con encabezado institucional y tabla de indicadores.
5. Vistas funcionales por rol	Implementada	Landing pública, inicio de sesión, home del vecino con modal de reporte, bandeja del funcionario con gestión de alertas y auditoría, y home del administrador. Operan sobre datos reales

		provenientes de la API.
6. Capa de servicios con contratos asíncronos	Implementada	Servicios (heatmap, sensores, alertas, indicadores y reportes) con firma asíncrona, listos para reemplazo directo por llamadas HTTP.
7. Diseño y documentación del backend	Implementada	Arquitectura, modelo de datos PostGIS, contrato HTTP con endpoints y subsistema IoT especificados en la documentación técnica.
8. API REST del backend (FastAPI)	Implementada	Endpoints REST sobre arquitectura hexagonal lite operativos; sirven datos al frontend y reciben la telemetría del subsistema IoT.
9. SQL definitivo	Implementada	Esquema PostGIS aplicado en Supabase: tablas, índices GIST, triggers y funciones RPC en uso del sistema.
10. Integración frontend-backend y autenticación	Implementada	El front consume la API real: autenticación por correo y contraseña via Supabase Auth y manejo uniforme de errores en funcionamiento.
11. Subsistema IoT (firmware e ingestor MQTT)	Implementada	Firmware del ESP32 publicando lecturas de dB via MQTT e ingestor del backend suscrito e insertando en la base de datos. Hay un sensor físico; la solución escala a más unidades sin cambios de código.
12. Cobertura de pruebas automatizadas	En progreso	Suites de pytest (251 casos, 96% de cobertura) en el backend y Vitest (32) + Cypress (31) en el frontend, todas en verde; cobertura del frontend en 93% de sentencias. El backend ya ejecuta pytest en CI; pendiente incorporar el frontend al pipeline de CI.

Integraciones necesarias

El funcionamiento de 40dB depende de varias integraciones con servicios externos. A continuación, se describe cada una, indicando que datos entran y salen y las consideraciones de seguridad aplicables.

Autenticación (Supabase Auth)

- **Que hace:** gestiona el registro y el inicio de sesión de los usuarios.
- **Datos:** el frontend envía las credenciales a Supabase Auth y recibe un token JWT, el backend verifica ese token de forma local, sin realizar un viaje y vuelta al servicio.
- **Decisión de alcance:** el MVP utiliza autenticación por correo electrónico y contraseña a través de Supabase auth.
- **Seguridad:** el frontend usa la anon_key solo para registro e inicio de sesión, la service_role_key se mantiene exclusivamente en el backend, el token JWT se verifica con un secreto compartido.

Base de datos gestionada (Supabase / PostgreSQL con PostGIS)

- **Que hace:** persiste usuarios, sensores, lecturas y reportes e historial de estados.
- **Datos de entrada:** el backend, único cliente de la base de datos, lee y escribe usando la service_role_key.
- **Seguridad:** todas las tablas tiene RLS habilitada, el frontend nunca accede directamente a las tablas de negocio.

Broker MQTT (HiveMQ Cloud)

- **Que hace:** provee el canal de telemetría IoT.
- **Datos de entrada:** los sensores ESP32 publican lecturas de dB, en mensaje JSON con el nivel medido y su timestamp en un topic por sensor, el backend esta suscrito a estos topics e inserta las lectura.
- **Seguridad:** TLS obligatorio en el puerto 8883 y credenciales por dispositivo.
- **Límites:** se utiliza la capa gratuita de HiveMQ, suficiente para el volumen del prototipo.

Cartografía base (OpenStreetMap via Leaflet)

- **Que hace:** provee las “tiles” del mapa sobre el que se dibuja el heatmap.
- **Datos de salida:** el cliente solicita las imágenes de las teselas al servidor de OpenStreetMap.
- **Consideración:** se mantiene la atribución correcta de OpenStreetMap y se respeta su política de uso de “tiles”.

Consideraciones transversales y pruebas

De forma transversal, todos los secretos se gestionan mediante variables de entorno y nunca se versionan, todos los canales usan TLS, y la API aplica CORS.

Estrategia y enfoque de calidad

Para 40dB, **calidad** no significa únicamente la ausencia de errores, sino el grado en que la plataforma satisface las necesidades declaradas e implícitas de sus tres tipos de usuario (ciudadano, municipalidad y administrador) que el sistema haga lo que debe, que no falle al cruzar datos ciudadanos con telemetría IoT, que proteja la información y que sea mantenible a medida que el prototipo evoluciona. Tomamos como vocabulario común el modelo de la norma **ISO/IEC 25010:2023** y conectamos cada esfuerzo de prueba con la característica de calidad que busca proteger.

Atributos de calidad priorizados

No todas las características pesan igual en un MVP de monitoreo acústico. Las priorizamos así:

- **Adecuación funcional (prioridad alta):** que crear un reporte, cambiar su estado, registrar una lectura o gestionar usuarios se comporte exactamente como define la regla de negocio. Es el núcleo del producto.
- **Fiabilidad (alta):** que la máquina de estados de un reporte (En espera → En atención → Atendido/Descartado) nunca permita transiciones inválidas, y que el ingestor MQTT siga vivo y resiliente ante mensajes corruptos.
- **Seguridad (alta):** verificación local del JWT, control de acceso por rol y por comuna, y validación de toda entrada del usuario en el servidor (nunca confiar en el cliente).
- **Mantenibilidad (media-alta):** la arquitectura por capas (dominio, aplicación, infraestructura, API) permite probar cada pieza en aislamiento; las pruebas automatizadas son el habilitador de futuros cambios sin romper lo existente.
- **Eficiencia de desempeño (media):** tiempos de respuesta razonables del API bajo concurrencia moderada, acorde al volumen de un prototipo comunal.
- **Capacidad de interacción / usabilidad (media):** validada de forma funcional a través de los flujos E2E del frontend (navegación, formularios, guardas de ruta).

Niveles y tipos de prueba

Aplicamos las pruebas de lo más pequeño a lo más completo, siguiendo el espíritu de la pirámide de pruebas pero con un peso importante en la capa de integración, con una API donde gran parte del valor está en cómo colaboran las capas. El orden es: **(1) unitarias** sobre lógica de dominio pura, **(2) humo** para confirmar que el API arranca y responde, **(3) integración** entre ruta + caso de uso + repositorio, **(4) funcionales / E2E** sobre los flujos completos de cada rol, **(5) regresión** para congelar bugs ya corregidos, **(6) seguridad** (estática de dependencias y dinámica de entrada) y **(7) carga** para medir el comportamiento bajo concurrencia.

Enfoque, datos y ambiente

El enfoque es enteramente automatizado y abarca los dos repositorios del sistema. El backend (40db-backend, API FastAPI) se prueba con pytest y el TestClient de FastAPI, que ejercita la aplicación ASGI en memoria sin abrir un puerto de red; las dependencias externas (Supabase, MQTT, almacenamiento) se sustituyen por mocks con unittest.mock, de modo que las pruebas son rápidas, repetibles y deterministas. El frontend (40db-frontend, Vue 3 + TypeScript + Vite + Pinia) se prueba en dos niveles: Vitest (sobre jsdom,

con @vue/test-utils y Pinia) para unitarias, integración, regresión y seguridad dinámica; y Cypress para humo y E2E sobre la app construida, inyectando una sesión falsa (cy.loginAs) y un backend simulado (cy.mockBackend) con fixtures JSON. A esto se suma una base de calidad estática (oxlint + ESLint, Prettier y verificación de tipos con vue-tsc) y la auditoría de dependencias (npm audit). Las pruebas se ejecutan en el ambiente local de desarrollo y están preparadas para integrarse a un pipeline de CI; el comando npm run verify encadena tipos, lint, pruebas y auditoría en una sola corrida.

Definición de "Hecho" (Definition of Done)

Una funcionalidad se considera validada cuando: (a) cuenta con pruebas automatizadas que cubren su camino feliz y sus errores esperados; (b) todas esas pruebas pasan en verde; (c) no tiene defectos abiertos de severidad alta o crítica; (d) la auditoría de dependencias no reporta vulnerabilidades altas o críticas sin mitigar; y (e) su comportamiento y evidencia quedan documentados en la matriz de pruebas.

Plan de pruebas

El plan de pruebas organiza todo el esfuerzo de verificación de 40dB. A continuación se presenta el resumen; el documento completo se adjunta como anexo.

Alcance de las pruebas

Se prueba de forma exhaustiva el **backend** (API FastAPI): la lógica de dominio pura, los casos de uso de la capa de aplicación, los repositorios y adaptadores de infraestructura (Supabase, almacenamiento, ingestor MQTT) y las rutas HTTP con su control de acceso por rol y comuna. En el **frontend** se prueban los flujos de usuario de los tres roles mediante Cypress.

Quedan **fuera de alcance** y se asume el riesgo correspondiente:

1. La verificación contra una instancia real de Supabase y del broker MQTT en producción se sustituyen por **mocks** y **fixtures**, por lo que no se valida la conectividad real ni la configuración de RLS en vivo
2. El firmware de los sensores ESP32, que es responsabilidad de hardware
3. Las teselas de OpenStreetMap, servicio de terceros. El riesgo se mitiga con pruebas de integración a nivel de adaptador y con pruebas de humo que confirman el contrato esperado de cada dependencia.

Elementos y funcionalidades a probar

Desde el punto de vista del usuario: registro e inicio de sesión y guardas de ruta por rol; creación, listado, detalle y cambio de estado de reportes; búsqueda de evidencia IoT; comentarios internos/externos; gestión de sensores (CRUD); panel de calor (heatmap); resumen horario de lecturas; gestión de usuarios (listar, promover, activar/desactivar) y archivos administrativos. Como **funcionalidades de regresión** se vuelve a probar, ante cada cambio, la máquina de estados de reportes y la normalización/validación de entradas, por ser las áreas de mayor riesgo de reaparición de defectos.

Criterios de aceptación o rechazo

Se consideran superadas las pruebas cuando se ejecuta el 100% de la suite automatizada con **0 fallos**, se alcanza una cobertura de al menos **80% en las capas de lógica de negocio** ("application/" y "domain/"), la auditoría de dependencias no arroja vulnerabilidades altas o críticas, y las pruebas de carga

cumplen sus umbrales (p95 y tasa de error). Se rechaza la entrega si existe cualquier prueba en rojo o un defecto de severidad alta sin resolver.

Criterios de suspensión y reanudación

Las pruebas se **suspenden** si una prueba de humo falla (el API no arranca o no responde lo mínimo), ya que no tiene sentido continuar; o si un defecto bloqueante impide ejecutar una suite completa. Se **reanudan** una vez corregido el bloqueo y re ejecutada con éxito la prueba de humo.

Premisas, dependencias y riesgos

Se asume que el entorno dispone de Node.js y Python con las dependencias instaladas y las variables de entorno de prueba configuradas. La principal dependencia es con el propio desarrollo: las pruebas evolucionan junto al código. El riesgo más relevante es que los **mocks** se desincronicen del comportamiento real de Supabase; se mitiga manteniendo pruebas de integración a nivel de adaptador y revisando los contratos ante cada cambio de esquema.

Actividad	J. Meléndez	I. Saavedra
Pruebas unitarias y de integración — backend	C	R/A
Pruebas de seguridad — backend (test_security.py)	C	R/A
Pruebas de carga (k6)	I	R/A
Pruebas unitarias / integración — frontend (Vitest)	R/A	C
Pruebas E2E — frontend (Cypress)	R/A	C
Pruebas de regresión (BUG-001, XSS)	R	A
Auditoría de dependencias (pip-audit / npm audit)	R	A
Consolidación de evidencias y matriz	R	C
Aprobación final del informe / plan	A	A

ⁱ Revisar Anexo “Plan de pruebas - 40dB.xlsx” en el envío del proyecto.

Tipos de pruebas aplicadas

A continuación se describe cómo se aplicó cada tipo de prueba al proyecto 40dB: qué se probó, con qué herramienta y por qué era pertinente.

Pruebas unitarias

Verifican la lógica de dominio pura en aislamiento total, sin red ni base de datos. Con **pytest** probamos la jerarquía de excepciones del dominio (que cada error mapee al código y estado HTTP correcto: 401, 403, 404, 503), la **máquina de estados** de los reportes (que "En espera" pueda pasar a "En atención" o "Descartado" pero no salte directo a "Atendido", y que los estados terminales no tengan transiciones de salida), la codificación/decodificación de los cursores de paginación y la validación de parámetros del heatmap (buckets permitidos y ventana máxima de tiempo). También se prueba unitariamente la decodificación del JWT (algoritmo soportado, token expirado, firma inválida). Son la base de la pirámide: rápidas, baratas y precisas.

Pruebas de humo (smoke)

Confirman que el API "está viva" tras cada cambio: que **todos** los endpoints responden con el **status code** esperado y no arrojan errores 500 inesperados, sin validar lógica de negocio. Con "TestClient" recorremos health, catálogos, reportes, heatmaps, usuarios, sensores, lecturas y archivos admin; además verificamos que los endpoints protegidos rechazan peticiones sin token (401/403), que el **middleware** de Correlation-Id propaga el identificador y que el formato de error es consistente. Es el filtro de entrada: si el humo falla, no se ejecuta el resto.

Pruebas de integración

Validan que varias capas colaboran correctamente. A nivel de **caso de uso**, probamos que al crear un reporte, cambiar su estado o promover un usuario, la ruta, el caso de uso, el validador y el repositorio se entienden entre sí (por ejemplo, que un título inválido devuelve 400 y no toca el servicio, o que una transición de estado inválida es rechazada). A nivel de **infraestructura**, probamos los repositorios contra un cliente Supabase simulado (camino feliz, resultado vacío y error que se traduce en "ExternalServiceError"), el almacenamiento de archivos (subida, **signed URL** y borrado con su compensación ante fallo de BD) y la decodificación de RPC. Además, una suite de integración dedicada ejercita el flujo completo de la API (ruta → caso de uso → repositorio) con todas las capas reales, mockeando únicamente el cliente de Supabase (get_supabase). Detectan errores de contrato entre capas que una unitaria no vería.

Pruebas funcionales y de extremo a extremo (E2E)

Recorren flujos completos como lo haría un usuario real. Con **Cypress** backend simulado con "cy.mockBackend()" y sesión inyectada con "cy.loginAs()" cubrimos:

las **guardas de ruta** (rutas protegidas que redirigen a “/auth” sin sesión y guardas por rol), el flujo del **vecino** (panel, abrir el modal de reporte, validación del formulario, ver reportes propios y abrir el detalle), el flujo del **funcionario** (ver KPIs, tabla de reportes de su comuna, filtrar por estado, abrir detalle, comentar y cambiar el estado a "En atención" de punta a punta) y el flujo del **administrador** (dashboard, KPIs de sensores, accesos rápidos y navegación a las secciones de gestión). Es importante precisar que estos E2E ejercitan el frontend de punta a punta **con el backend mockeado** la lógica del backend se valida de forma independiente con la suite de pytest.

Pruebas de regresión

Aseguran que un defecto ya corregido no reaparezca tras nuevos cambios. La máquina de estados y la validación/normalización de entradas (las áreas de mayor riesgo) ya cuentan con pruebas que actúan como "guardianes": cualquier cambio que vuelva a permitir una transición inválida o que deje pasar una entrada no normalizada falla de inmediato.

Pruebas de seguridad

Verifican la autenticación, la autorización y el manejo seguro de la entrada en el borde HTTP. En el backend, la suite de seguridad (47 casos con pytest) agrupa cinco frentes: autenticación obligatoria (ningún endpoint protegido responde sin token: 401/403); resistencia del JWT a ataques (token con alg=none, firma inválida, expirado, audiencia incorrecta, algoritmo no soportado, sin sub o basura, probados contra GET /usuarios/me, que falla antes de tocar la base de datos); autorización por roles (un rol inferior no alcanza endpoints de rol superior, 403, ejercitando la cadena real de decodificar el JWT, cargar el usuario y aplicar el control de rol); usuario inhabilitado (token válido de un usuario inactivo o inexistente, 401); y no fuga de información (un error interno no expone trazas ni detalles). En el frontend, las pruebas de entrada (4 casos con Vitest) confirman que títulos y mensajes con <script> o se muestran como texto y no como HTML, como defensa ante XSS. De forma transversal, la seguridad de dependencias se audita con npm audit (frontend) y pip-audit (backend).

Pruebas de carga y rendimiento

Miden el comportamiento del backend bajo concurrencia. Con k6 se definen dos escenarios sobre la API: un humo de carga que siempre se ejecuta (GET /health/live y GET /openapi.json, que miden el techo del stack ASGI sin tocar Supabase) y un escenario de API autenticada opcional (/usuarios/me, /sensores y /reportes/mios contra Supabase real, cuando se entrega un token). La corrida falla si no se cumplen los umbrales: tasa de peticiones fallidas menor al 1%, percentil 95 del tiempo de respuesta bajo 500 ms y tasa de éxito de los checks sobre 99%. La línea base local (20 usuarios virtuales durante 45 s, 2960 verificaciones) arrojó p95 = 7,74 ms en el tiempo de respuesta HTTP, 0% de peticiones fallidas y 100% de checks exitosos. Estas pruebas no corren en la integración continua porque requieren el servidor realmente levantado.

Herramientas y software utilizados

Seleccionamos herramientas adecuadas al stack del proyecto (FastAPI/Python en 40db-backend, Vue 3 + TypeScript + Vite en 40db-frontend). La siguiente tabla resume cada herramienta, su uso, versión y la razón de su elección.

Herramienta	Tipo de prueba / Uso		Justificación
@vue / test-utils	Montaje de componentes Vue		Navegador real. Utilidades oficiales para mount, eventos (v-model), props y accesibilidad.
Pinia (test)	Aislamiento de estado		createPinia + setActivePinia dan un store limpio por prueba.
Cypress	Humo y E2E (frontend)		Pruebas en navegador real sobre el preview sesión falsa + cy.intercept() para mockear el backend.
start-server-and-test	Orquestación E2E		Levanta el preview y dispara Cypress en una sola orden.
npm audit	Seguridad estática (SCA, deps. frontend)		--omit=dev --audit-level=high. Detecta CVE conocidos en dependencias.
pip-audit	Seguridad estática (SCA, deps.backend)		Equivalente de npm audit para Python.
k6 (Grafana)	Carga / rendimiento (contra el backend)		Escenarios de carga en JS; mide p95, tasa de error y throughput.
oxlint + ESLint	Análisis estático		Base de la pirámide de calidad: detecta errores y malos patrones.

Prettier	Formato		Estilo de código consistente.
vue-tsc	Verificación de tipos		type-check sobre todo el proyecto TypeScript.
GitHub (Issues / Projects / Actions)	Gestión y CI		Seguimiento de incidencias, trazabilidad de commits (regresión) y ejecución de la suite.

Ejecución de pruebas y resultados

Esta sección muestra qué se obtuvo al ejecutar la suite de pruebas y se respalda con evidencia. El backend se ejecutó por completo con pytest (corrida del 2026-06-21): 251 pruebas en verde y 96% de cobertura sobre app/. El frontend (Vitest y Cypress) suma 63 pruebas en verde (32 con Vitest y 31 con Cypress). En total, las 314 pruebas automatizadas del sistema pasan sin fallos. Las capturas que respaldan cada cifra se incluyen al final de esta sección.

Resumen por tipo de prueba (sistema completo)

Tipo de prueba	N. de casos	Estado	Herramientas
Unitarias	29 (backend 13 + frontend 16)	29/29 en verde	pytest / Vitest
Humo (smoke)	45 (backend 43 + frontend 2)	45/45 en verde	TestClient / Cypress
Integración	155 (backend 148 + frontend 7)	155/155 en verde	pytest / Vitest + @vue/test-utils
Funcionales / E2E	29 (frontend)	29/29 en verde	Cypress
Regresión	5 (frontend, BUG-001)	5/5 en verde	Vitest
Seguridad	51 (backend 47 + frontend 4) + auditoría de deps.	51/51 en verde	pytest / Vitest + npm audit / pip-audit
Carga	2 escenarios	Humo de carga OK: p95 ≈ 17 ms, 0% errores	k6

Detalle - Backend (40db-backend, pytest)

Resultado de la corrida con pytest: 251 pruebas en verde y 96% de cobertura sobre app/ (7,81 s). Desglose por suite:

- Lógica de dominio pura: 13 casos (jerarquía de errores, máquina de estados, cursor, validación de heatmap).
- Casos de uso (capa de aplicación): 69 casos (reportes, sensores, usuarios, comentarios, archivos, heatmap, resumen horario).
- Infraestructura (repositorios, almacenamiento, RPC, ingestor MQTT): 11 funciones que agrupan decenas de escenarios.
- Rutas HTTP y manejadores de error: 36 casos (control de acceso por rol y comuna, mapeo de excepciones).
- Humo: 43 casos (todos los endpoints + 401/403 sin token + Correlation-Id + formato de error).
- Integración de extremo a extremo: 32 casos (flujo completo de la API → caso de uso → repositorio con todas las capas reales, mockeando únicamente el cliente de Supabase).
- Seguridad: 47 casos (autenticación obligatoria, resistencia del JWT a ataques, autorización por roles, usuario inhabilitado y no fuga de información).

tests coverage				
coverage: platform win32, python 3.13.9-final-0				
Name	Stmts	Miss	Cover	Missing
app__init__.py	0	0	100%	
app\api__init__.py	0	0	100%	
app\api\deps.py	70	8	89%	24-26, 39-43
app\api\error_handlers.py	29	3	90%	20-23
app\api\middleware__init__.py	0	0	100%	
app\api\middleware\correlation.py	10	0	100%	
app\api\routes__init__.py	0	0	100%	
app\api\routes\catalogos.py	11	0	100%	
app\api\routes\health.py	25	1	96%	34
app\api\routes\heatmaps.py	19	2	89%	22-23
app\api\routes\lecturas.py	19	2	89%	17-18
app\api\routes\reportes.py	70	1	99%	104
app\api\routes\reportes_admin_archivos.py	30	0	100%	
app\api\routes\sensores.py	32	0	100%	
app\api\routes\usuarios.py	46	0	100%	
app\api\schemas__init__.py	0	0	100%	
app\api\schemas\lectura.py	18	0	100%	
app\api\schemas\reporte.py	50	0	100%	
app\api\schemas\reporte_archivo_admin.py	20	0	100%	
app\api\schemas\reporte_comentario.py	34	3	91%	27, 34, 36
app\api\schemas\sensor.py	44	1	98%	51
app\api\schemas\usuario.py	24	0	100%	
app\application__init__.py	0	0	100%	
app\application\buscar_evidencia.py	6	0	100%	
app\application\cambiar_activo_usuario.py	17	0	100%	
app\application\cambiar_estado_reporte.py	27	0	100%	
app\application\crear_reporte.py	26	1	96%	58
app\application\listar_usuarios.py	13	0	100%	
app\application\obtener_heatmap.py	17	0	100%	
app\application\obtener_resumen_horario.py	22	0	100%	
app\application\promover_usuario.py	39	0	100%	
app\application\registrar_lectura.py	4	0	100%	
app\application\reporte_comentarios.py	42	1	98%	31
app\application\reportes_admin_archivos.py	70	3	96%	50, 117-118
app\application\sensores.py	51	0	100%	
app\core__init__.py	0	0	100%	
app\core\config.py	29	1	97%	51
app\core\supabase_client.py	6	0	100%	
app\domain__init__.py	0	0	100%	
app\domain\entities.py	79	0	100%	
app\domain\errors.py	35	0	100%	
app\domain\ports.py	7	7	0%	1-26
app\infrastructure__init__.py	0	0	100%	
app\infrastructure\db__init__.py	0	0	100%	
app\infrastructure\db\lectura_repo.py	43	0	100%	
app\infrastructure\db\reporte_archivo_admin_repo.py	65	2	97%	136-137
app\infrastructure\db\reporte_comentario_repo.py	29	0	100%	
app\infrastructure\db\reporte_repo.py	85	5	94%	11-12, 96, 136, 187
app\infrastructure\db\resumen_horario_repo.py	24	1	96%	16
app\infrastructure\db\rpc.py	18	0	100%	
app\infrastructure\db\sensor_repo.py	100	14	86%	33, 37-38, 52, 66-67, 79-80, 84-86, 147, 149, 153
app\infrastructure\db\usuario_repo.py	89	0	100%	
app\infrastructure\mqtt__init__.py	0	0	100%	
app\infrastructure\mqtt\ingestor.py	66	0	100%	
app\infrastructure\storage__init__.py	0	0	100%	
app\infrastructure\storage\reportes_admin_storage.py	27	0	100%	
app\main.py	37	12	68%	18-33
app\patterns__init__.py	0	0	100%	
TOTAL	1624	68	96%	
Coverage HTML written to dir htmlcov				
251 passed, 6 warnings in 7.81s				

Coverage report: 96%

Files Functions Classes

coverage.py v7.14.2, created at 2026-06-21 12:45 -0400

File ▲	statements	missing	excluded	coverage
app__init__.py	0	0	0	100%
app\api__init__.py	0	0	0	100%
app\api\deps.py	70	8	0	89%
app\api\error_handlers.py	29	3	0	90%
app\api\middleware__init__.py	0	0	0	100%
app\api\middleware\correlation.py	10	0	0	100%
app\api\routes__init__.py	0	0	0	100%
app\api\routes\catalogos.py	11	0	0	100%
app\api\routes\health.py	25	1	0	96%
app\api\routes\heatmaps.py	19	2	0	89%
app\api\routes\lecturas.py	19	2	0	89%
app\api\routes\reportes_admin_archivos.py	30	0	0	100%
app\api\routes\reportes.py	70	1	0	99%
app\api\routes\sensores.py	32	0	0	100%
app\api\routes\usuarios.py	46	0	0	100%
app\api\schemas__init__.py	0	0	0	100%
app\api\schemas\lectura.py	18	0	0	100%
app\api\schemas\reporte_archivo_admin.py	20	0	0	100%
app\api\schemas\reporte_comentario.py	34	3	0	91%
app\api\schemas\reporte.py	50	0	0	100%
app\api\schemas\sensor.py	44	1	0	98%
app\api\schemas\usuario.py	24	0	0	100%
app\application__init__.py	0	0	0	100%
app\application\buscar_evidencia.py	6	0	0	100%
app\application\cambiar_activo_usuario.py	17	0	0	100%
app\application\cambiar_estado_reporte.py	27	0	0	100%
app\application\crear_reporte.py	26	1	0	96%
app\application\listar_usuarios.py	13	0	0	100%
app\application\obtener_heatmap.py	17	0	0	100%
app\application\obtener_resumen_horario.py	22	0	0	100%
app\application\promover_usuario.py	39	0	0	100%
app\application\registrar_lectura.py	4	0	0	100%
app\application\reporte_comentarios.py	42	1	0	98%
app\application\reportes_admin_archivos.py	70	3	0	96%
app\application\sensores.py	51	0	0	100%
app\core__init__.py	0	0	0	100%
app\core\config.py	29	1	0	97%

app\application\cambiar_estado_reporte.py	27	0	0	100%
app\application\crear_reporte.py	26	1	0	96%
app\application\listar_usuarios.py	13	0	0	100%
app\application\obtener_heatmap.py	17	0	0	100%
app\application\obtener_resumen_horario.py	22	0	0	100%
app\application\promover_usuario.py	39	0	0	100%
app\application\registrar_lectura.py	4	0	0	100%
app\application\reporte_comentarios.py	42	1	0	98%
app\application\reportes_admin_archivos.py	70	3	0	96%
app\application\sensores.py	51	0	0	100%
app\core__init__.py	0	0	0	100%
app\core\config.py	29	1	0	97%
app\core\supabase_client.py	6	0	0	100%
app\domain__init__.py	0	0	0	100%
app\domain\entities.py	79	0	0	100%
app\domain\errors.py	35	0	0	100%
app\domain\ports.py	7	7	38	0%
app\infrastructure__init__.py	0	0	0	100%
app\infrastructure\db__init__.py	0	0	0	100%
app\infrastructure\db\lectura_repo.py	43	0	0	100%
app\infrastructure\db\reporte_archivo_admin_repo.py	65	2	0	97%
app\infrastructure\db\reporte_comentario_repo.py	29	0	0	100%
app\infrastructure\db\reporte_repo.py	85	5	0	94%
app\infrastructure\db\resumen_horario_repo.py	24	1	0	96%
app\infrastructure\db\rpc.py	18	0	0	100%
app\infrastructure\db\sensor_repo.py	100	14	0	86%
app\infrastructure\db\usuario_repo.py	89	0	0	100%
app\infrastructure\mqtt__init__.py	0	0	0	100%
app\infrastructure\mqtt\ingestor.py	66	0	0	100%
app\infrastructure\storage__init__.py	0	0	0	100%
app\infrastructure\storage\reportes_admin_storage.py	27	0	0	100%
app\main.py	37	12	0	68%
app\patterns__init__.py	0	0	0	100%
Total	1624	68	38	96%

```
PS C:\Users\del\Documents\40 db prueba 3\40db-backend> .\.venv\Scripts\ruff.exe check app/ tests/
All checks passed!
PS C:\Users\del\Documents\40 db prueba 3\40db-backend>
```

```
PS C:\Users\del\Documents\40 db prueba 3\40db-frontend> cd ..
PS C:\Users\del\Documents\40 db prueba 3> cd .\40db-backend\
PS C:\Users\del\Documents\40 db prueba 3\40db-backend> .\.venv\Scripts\python.exe -m pip_audit
No known vulnerabilities found
PS C:\Users\del\Documents\40 db prueba 3\40db-backend>
```

Detalle - Frontend Vitest (40db-frontend)

Resultado: 6 archivos / 32 casos / 32 passed (evidencias/test-unit.txt):

- errors.spec.ts — unitaria — 6 · heatmap-features.spec.ts — unitaria — 10
- BaseInput.integration.spec.ts — integración — 4 · toast.integration.spec.ts — integración — 3
- regression.estado-reporte.spec.ts — regresión (BUG-001) — 5 · security.input.spec.ts — seguridad/XSS — 4

```
> 40db-frontend@0.0.0 test:unit:run
> vitest run

v4.1.4 C:/Users/del/Documents/40 db prueba 3/40db-frontend

✓ src/__tests__/errors.spec.ts (6 tests) 10ms
✓ src/__tests__/regression.estado-reporte.spec.ts (5 tests) 11ms
✓ src/__tests__/heatmap-features.spec.ts (10 tests) 13ms
(node:19236) Warning: '--localstorage-file' was provided without a valid path
(Use 'node --trace-warnings ...' to show where the warning was created)
✓ src/__tests__/BaseInput.integration.spec.ts (4 tests) 51ms
✓ src/__tests__/security.input.spec.ts (4 tests) 54ms
✓ src/__tests__/toast.integration.spec.ts (3 tests) 77ms

Test Files 6 passed (6)
Tests 32 passed (32)
Start at 12:48:04
Duration 11.61s (transform 1.93s, setup 0ms, import 3.06s, tests 217ms, environment 59.97s)
```

All files

93.18% Statements 82/88 89.23% Branches 58/65 79.31% Functions 23/29 92.5% Lines 74/80

Press n or j to go to the next uncovered block, b, p or k for the previous block.

Filter:

File	Statements	
components/common		76.26% 16/23
services		100% 12/13
stores		93.33% 14/15
types		100% 5/5
utils		100% 32/32

Branches	Functions	Lines
06.11% 21/36	50% 5/10	76.19% 16/21
100% 9/9	100% 6/6	100% 13/13
87.5% 7/8	83.33% 5/6	92.3% 12/13
100% 0/0	100% 1/1	100% 5/5
91.66% 11/12	100% 5/5	100% 28/28

```
PS C:\Users\del\Documents\40 db prueba 3\40db-frontend> npm run type-check
> 40db-frontend@0.0.0 type-check
> vue-tsc --build
```

```
# npm audit report
dompurify <=3.4.10
Severity: moderate
DOMPurify: 'IN_PLACE' mode trusts attacker-controlled 'nodeName' on live non-form nodes, allowing script retention and XSS via attacker-supplied DOM objects - https://github.com/advisories/GHSA-xbvx-rjvf-j5pl
DOMPurify: Hook mutation of 'data.allowedTags' / 'data.allowedAttributes' permanently pollutes 'DEFAULT_ALLOWED_TAGS' / 'DEFAULT_ALLOWED_ATTR' - https://github.com/advisories/GHSA-76mc-f452-cxcn
DOMPurify: Cross-realm IN_PLACE sanitization leaves executable markup intact via realm-bound 'instanceof' checks - https://github.com/advisories/GHSA-hpcv-96wg-7vj8
DOMPurify: IN_PLACE mode preserves attributes of a clobbered root element, allowing XSS via attacker-controlled root DOM - https://github.com/advisories/GHSA-r47g-fvhr-h676
DOMPurify: Trusted Types policy survives 'clearConfig()' and can poison later 'RETURN_TRUSTED_TYPE' output - https://github.com/advisories/GHSA-vxr8-fq3u-vva9
DOMPurify: SAFE_FOR_TEMPLATES bypass - template expressions survive sanitization inside <template> content when using DOM output modes - https://github.com/advisories/GHSA-gvmj-q25r-r7wr
DOMPurify: IN_PLACE Sanitization Bypass via Attached Shadow Root Inside <template> content - https://github.com/advisories/GHSA-rp9w-3fw7-7cmq
DOMPurify: Permanent 'ALLOWED_ATTR' pollution via 'setConfig()' bypassing the hook clone-guard (incomplete fix of the 3.4.7 hook-pollution patch) - https://github.com/advisories/GHSA-cmmh-pvxp-8882
fix available via `npm audit fix`
node_modules/dompurify
! moderate severity vulnerability
```

Detalle - Frontend Cypress (40db-frontend)

Resultado: 5 specs / 31 casos / 31 passed (All specs passed, evidencias/e2e.txt):

```
=====
```

(Run Finished)						
Spec		Tests	Passing	Failing	Pending	Skipped
✓ 02_auth.cy.ts	00:08	9	9	-	-	-
✓ 03_vecino.cy.ts	00:11	6	6	-	-	-
✓ 04_funcionario.cy.ts	00:07	7	7	-	-	-
✓ 05_admin.cy.ts	00:06	7	7	-	-	-
✓ home.cy.ts	00:01	2	2	-	-	-
✓ All specs passed!	00:34	31	31	-	-	-

```
I GET /health/live: status 200
I GET /openapi.json: status 200

CUSTOM
check_success_rate.....: 100.00% 2960 out of 2960

HTTP
http_req_duration.....: avg=5.92ms   min=3.03ms   med=5.67ms   max=152.96ms p(90)=7.22ms p(95)=7.74ms
{ expected_response:true }...: avg=5.92ms   min=3.03ms   med=5.67ms   max=152.96ms p(90)=7.22ms p(95)=7.74ms
http_req_failed.....: 0.00%    0 out of 2960
http_reqs.....: 2960    65.223062/s

EXECUTION
iteration_duration.....: avg=512.29ms min=508.21ms med=511.96ms max=806.28ms p(90)=513.88ms p(95)=514.63ms
iterations.....: 1480    32.611531/s
vus.....: 2    min=2    max=20
vus_max.....: 20    min=20    max=20

NETWORK
data_received.....: 49 MB    1.1 MB/s
data_sent.....: 253 kB    5.6 kB/s

Running (0m45.4s), 00/20 VUs, 1480 complete and 0 interrupted iterations
default I [=====] 00/20 VUs  45s

C:\Users\del\Documents\40_db_prueba_3\40db-backend>
```

- home.cy.ts (humo) 2 · 02_auth.cy.ts (guardas + login) 9 · 03_vecino.cy.ts 6 · 04_funcionario.cy.ts 7 · 05_admin.cy.ts 7.

En conjunto, el **frontend** suma **63/63 casos en verde** (32 Vitest + 31 Cypress).

Indicadores y métricas

- **Casos exitosos:** frontend 63/63 en verde (Vitest 32/32 + Cypress 31/31) y backend 251/251 en verde con pytest; type-check y lint de src sin errores. En total, las 314 pruebas automatizadas pasan sin fallos.
- **Seguridad de dependencias (frontend):** en el frontend, npm audit --omit=dev reporta 0 high/critical y 1 moderate (dompurify vía jspdf, ruta no crítica de exportación a PDF), documentada en lugar de forzar la actualización; en el backend, pip-audit no reporta vulnerabilidades conocidas.
- **Cobertura de código:** backend medida en 96% sobre app/ (pytest-cov: 1624 sentencias, 68 sin cubrir); frontend medida en 93,18% de sentencias y 92,5% de líneas (89,23% de ramas), con services, stores y utils sobre el 93%.
- **Carga:** ejecutada con k6 (humo de carga, 20 VUs durante 45 s); línea base local p95 = 7,74 ms en el tiempo de respuesta HTTP, 0% de errores y 2960/2960 checks exitosos. Cumple los umbrales definidos: error < 1%, p95 < 500 ms y éxito de checks > 99%.

Conclusión

La contaminación acústica es un problema creciente en las ciudades chilenas que carece de herramientas tecnológicas accesibles para su monitoreo continuo a nivel comunal. Las soluciones existentes —mapas de ruido modelados por software y registros de denuncias— son valiosas pero insuficientes para una gestión ambiental proactiva, ya que entregan diagnósticos estáticos o reactivos que no reflejan la realidad acústica en tiempo real ni permiten priorizar intervenciones con base en datos empíricos.

40dB aborda esta brecha mediante una plataforma web que combina la participación ciudadana con sensores IoT para generar datos acústicos continuos y geolocalizados. El mecanismo de cruce entre ambas fuentes —donde las lecturas de los sensores actúan como ground truth para validar los reportes de los vecinos— aporta un enfoque novedoso que balancea la cobertura amplia de la participación ciudadana con la confiabilidad técnica de los sensores fijos. Al cierre de esta evaluación, el sistema opera de extremo a extremo con datos reales: un sensor ESP32 publica sus lecturas vía MQTT al broker HiveMQ Cloud, el backend FastAPI las ingiere y las persiste en la base de datos Supabase con PostGIS, y el frontend Vue 3 consume esa información a través de la API REST para mostrarla en el heatmap interactivo y en las vistas diferenciadas por rol (vecino, funcionario municipal y administrador).

Los beneficios esperados de la solución son dotar a la municipalidad de un mapa de ruido vivo y permanentemente actualizado, empoderar al vecino con datos objetivos sobre su entorno acústico, reducir la dependencia de las denuncias como único mecanismo de detección y generar series históricas que permitan evaluar tendencias y el impacto de intervenciones a lo largo del tiempo. Los próximos pasos del semestre contemplan ampliar la cobertura de pruebas automatizadas, refinar el dashboard municipal con base en la retroalimentación recibida, completar la documentación operacional del despliegue y preparar la presentación final del piloto funcional para la Municipalidad de Maipú, dejando además sentadas las bases arquitectónicas —particionamiento mensual de la tabla de lecturas y agregación precomputada vía `pg_cron`— para que el sistema pueda escalar a una red de sensores más extensa sin necesidad de reescritura.

Bibliografía

Organización Mundial de la Salud. (2021). *WHO global air quality guidelines: particulate matter (PM_{2.5} and PM₁₀), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide*. <https://iris.who.int/server/api/core/bitstreams/4c6945b7-497a-4043-9e6f-98d6774b3b61/content>

Superintendencia del Medio Ambiente. (7 de agosto de 2024). *SMA publica nueva guía para tramitar Programas de Cumplimiento por infracciones a la Norma de Ruido*. Recuperado el 5 de abril de 2026 de <https://portal.sma.gob.cl/index.php/sma-publica-nueva-guia-para-tramitar-programas-de-cumplimiento-por-infracciones-a-la-norma-de-ruido/>

Ministerio del Medio Ambiente. (2024). *Mapas de ruido*. Ruido Ambiental. Recuperado el 5 de abril de 2026 de <https://ruido.mma.gob.cl/mapas-de-ruido/>

Superintendencia del Medio Ambiente. (s. f.). *Infraestructura de Datos Espaciales (IDE)*. Recuperado el 5 de abril de 2026 de <https://ide.sma.gob.cl/>

Ministerio del Medio Ambiente. (s. f.). *Redes de monitoreo en línea*. Sistema de Información Nacional de Calidad del Aire. Recuperado el 5 de abril de 2026 de <https://sinca.mma.gob.cl/index.php/redes>